LangChain: fundamentos prácticos

Máster AI Engineer · Sesión 4 · Ignacio de Pastors

thePower Tech School

AGENDA

01 Bloque teórico
02 Bloque práctico
03 Ejercicio para casa
04 Cierre y dudas



thePower
Tech School

01

Bloque teórico

thePower
Tech School

¿Qué es LangChain y por qué existe?

Framework open source para crear aplicaciones sobre LLMs.

Estandariza cómo **pedir**, dar **contexto** y **encadenar pasos**
Dejas de reinventar el pegamento en cada proyecto

Nació en **2022**; hoy es el **estándar de facto**
La documentación y los ejemplos que encontrarás lo asumen

El LLM solo hace una cosa: **texto entra, texto sale**
No recuerda, no consulta tus datos y no llama a APIs: eso lo montas alrededor

Caso real: **un asistente de RR. HH. sobre tus PDF**
Prompt + tus documentos + memoria del chat: eso es LangChain en 30 líneas

ANALOGÍA
Si el LLM es el motor, LangChain es el chasis, el volante y el salpicadero: **convierte el motor en un coche que puedes conducir**.
LangChain no sustituye al modelo: lo envuelve para que sea útil.
EN LA PRÁCTICA
Cambiar de OpenAI a Llama local es cambiar una pieza, no reescribir la app.
No compensa para un script de 20 líneas; compensa en cuanto hay varios pasos, memoria o documentos propios.

thePower Tech School

La arquitectura: los bloques de LangChain

Models
LLM y ChatModels
cambias de proveedor sin tocar el resto
Ej.: ChatOpenAI() hoy, ChatOllama() mañana, sin tocar nada más

Prompts
plantillas con variables
escribes la instrucción una vez, la rellenas mil
Ej.: 'Resume este {texto} en {n} frases'

Parsers
salida a datos usables
de texto suelto a un JSON o una lista
Ej.: de 'la respuesta es 42' al número 42

Chains / LCEL
componer con |
la salida de una pieza entra en la siguiente
Ej.: prompt | modelo | parser

Memory
contexto y memoria
por defecto el modelo no recuerda nada
Ej.: recordar que el usuario se llama Ignacio

Retrieval
RAG: tus datos
busca el fragmento y se lo pasa al modelo
Ej.: responder sobre el PDF de vacaciones de tu empresa

Es Lego: todas las piezas usan el **mismo conector**, por eso puedes cambiar una sin tocar las demás.

La mayoría de apps reales solo usan tres: Models, Prompts y Chains. Los agentes no son una séptima caja: son la capa que decide qué piezas usar.

thePower Tech School

Model I/O: cómo entra y sale la información

Tres piezas que se combinan:

PromptTemplate: plantilla con variables
reutilizas la instrucción sin reescribirla; ej.: 'Traduce {texto} al {idioma}'

Modelo: LLM o ChatModel
la misma llamada vale para cualquier proveedor: cambias ChatOpenAI por ChatOllama y ya

OutputParser: convierte la respuesta en datos
*el modelo **SIEMPRE** devuelve texto, aunque le pidas JSON; el parser lo convierte en un objeto real*

Ventaja: cada pieza se cambia por separado
si algo falla, sabes exactamente en cuál de las tres mirar

CÓDIGO

```
prompt = plantilla.format(tema='IA')
respuesta = modelo.invoke(prompt)
datos = parser.parse(respuesta)
```

Tres líneas, tres piezas: cambias una sin tocar las otras dos.
Caso de uso: clasificar 500 emails de soporte en categorías y volcarlas a un CSV.
Si el parser falla existen parsers con reintento que devuelven el error al modelo.

thePower Tech School

LLM vs ChatModel: dos formas de hablar con el modelo

LLM · clásico
Entrada y salida: **texto plano**
Para completar texto
Sin roles ni historial de conversación
En desuso en apps nuevas
Lo verás como OpenAI() en tutoriales antiguos
Ej.: le das 'Érase una vez' y sigue escribiendo el cuento

ChatModel · estándar hoy
Mensajes con rol: system, human, ai
Soporta herramientas (tools)
Base de **chatbots y agentes**
Mantiene el hilo de la conversación
El mensaje **system** define el personaje y las reglas
Es ChatOpenAI(): lo que usaremos hoy
Ej.: system='eres un asistente jurídico, responde breve y en español'

Sin ChatModel no hay **tools**, y sin tools no hay **agentes**. En duda, siempre **ChatModel**.

thePower Tech School

LCEL: componer piezas con el operador |

LangChain Expression Language.

Cada pieza es un **Runnable**
todas tienen el mismo enchufe: un prompt, un modelo, un parser y hasta una cadena entera son Runnables

El **|** las conecta como una **tubería**
la salida de una entra en la siguiente, sin variables intermedias ni pegamento a mano

Gratis: **invoke, stream, batch y async**
stream para que la respuesta aparezca palabra a palabra; batch para 100 entradas de golpe

Sustituye a **LLMChain**
recomendado desde LangChain v0.3; si ves LLMChain, es código antiguo

CÓDIGO
cadena = **prompt | modelo | parser**
cadena.invoke({'tema': 'IA'})
cadena.stream(...) cadena.batch([...]) await cadena.ainvoke(...)
Léelo en voz alta: relleno la plantilla, se la paso al modelo, parseo lo que vuelve.

thePower Tech School

Chains: de una cadena simple a una compuesta

SIMPLE

Un solo flujo: prompt | modelo | parser

Ejemplo: traducir un texto o clasificar un email

Una sola llamada al modelo

Entra algo, sale algo. Es la mayoría de los casos y por donde debes empezar.



COMPUESTA

Varias etapas encadenadas

La salida de una alimenta la siguiente

Cada etapa hace una sola tarea y sale mejor

Puedes inspeccionar el paso intermedio o cambiar solo una etapa

Más calidad, pero más llamadas, más coste y más latencia.



¿Por qué no pedírselo todo en un solo prompt? Porque el modelo hace las tres cosas regular. **Una etapa por cada tarea claramente distinta.**

thePower Tech School

Memoria: que el asistente recuerde la conversación

El LLM es **sin estado**: **no recuerda nada** entre llamadas.



Buffer: guarda todo el historial
fiel al detalle, pero crece y consume tokens hasta reventar la ventana de contexto

Summary: guarda un resumen
aguanta charlas largas, pero pierde detalle: los datos concretos se difuminan

En la práctica se combinan
los últimos mensajes literales más un resumen de todo lo anterior

El estado lo pones tú, alrededor
el modelo sigue siendo amnésico en cada llamada; lo que cambia es lo que le das de entrada

CÓMO SE HACE EN LANGCHAIN

RunnableWithMessageHistory envuelve tu cadena y guarda el historial por **session_id**: cada usuario tiene su propio hilo de conversación.
En cada llamada se le reenvía la conversación: por eso recordar cuesta tokens.
Si tienes mil usuarios, son mil session_id: cada hilo va por separado.
Se guarda en memoria del proceso, o en Redis o Postgres si quieres que sobreviva a un reinicio.
Ej.: dices “me llamo Ignacio” y dos mensajes después sigue sabiéndolo.

thePower Tech School

RAG: darle tus propios datos al modelo

1

1. Indexar
docs, chunks y embeddings
se hace una vez, offline
Ej.: 300 PDF partidos en trozos de 800 palabras

2

2. Preguntar
el usuario pregunta
también pasa a embedding
Ej.: '¿Cuántos días de vacaciones tengo?'

3

3. Recuperar
top-k fragmentos
busca significado, no palabras
Ej.: encuentra el párrafo de 'periodo vacacional'

4

4. Responder
el LLM usa ese contexto
responde solo con lo recuperado
Ej.: '23 días hábiles, según el convenio'

Sin RAG el modelo solo sabe lo que aprendió. **Con RAG responde sobre TUS documentos.**

Es un examen a libro abierto: el modelo no se lo aprende, consulta la página exacta antes de responder. El 80 % de la calidad está en cómo partes y recuperas, no en el LLM.

thePower Tech School

Agentes: el LLM razona y usa herramientas

Pensar
qué necesito hacer
razona sobre lo que le falta
Ej.: 'necesito las ventas del mes pasado'

Elegir tool
qué herramienta usar
de la lista que tú le das
Ej.: consultar la base de datos

Ejecutar
llama a la herramienta
lo ejecuta tu código
*el modelo **NO** ejecuta: solo pide la llamada*

Observar
lee el resultado
y vuelve a pensar
si falta algo, otra vuelta

Repite el ciclo (**patrón ReAct**) hasta poder dar la respuesta final.

Cadena es una **receta**; agente es un **cocinero**: tú das el objetivo y las herramientas, él decide el orden.
En producción, siempre con límite de iteraciones y presupuesto: pueden entrar en bucle o elegir mal la herramienta.

thePower Tech School

El ecosistema: más allá de LangChain

LangChain
las piezas y cadenas
models, prompts, chains, memoria
Responde a: ¿Cómo construyo las piezas y las uno?

LangGraph
agentes con estado
ciclos, ramas y control humano
Responde a: ¿Cómo controlo un agente que da vueltas? (human in the loop)

LangSmith
observabilidad y evals
cada llamada, coste y latencia
Responde a: ¿Por qué me ha respondido esto? Es la caja negra del avión

LangServe
cadena como API
publícala como API REST
Responde a: ¿Cómo lo publico? API REST en pocas líneas

Regla 2026: los agentes serios se construyen con **LangGraph** sobre las piezas de LangChain.

No hace falta usar los cuatro: para un prototipo, LangChain solo va sobrado. LangSmith se agradece cuando algo falla; LangGraph, cuando el flujo deja de ser una línea recta.

thePower Tech School

02

Bloque práctico

thePower
Tech School

Cómo vamos a trabajar: el ciclo con Claude Code

1

Pedir

el prompt en lenguaje natural

'qué quiero, no cómo'

2

Revisar

leer el código que genera

'tú firmas el código, no Claude'

3

Ejecutar

lanzarlo en la terminal

'el resultado real manda'

4

Entender

conectarlo con la teoría

'qué caja del mapa he usado'

Si no funciona, se vuelve a Pedir con lo aprendido. Cada paso de hoy tiene tres slides: **resultado, anatomía del código y una variación en vivo. Los prompts, en un fichero aparte.**

Stack de hoy 100 % gratis: Python + LangChain + Ollama (llama3.1 en local). Coste de API: 0 EUR.

thePower Tech School

Antes de empezar: el entorno en 4 piezas

1. **Python 3.10 o superior**
   con el lanzador py o python en el PATH
2. **Ollama**
   motor para correr modelos en vuestro portátil
3. **Modelo llama3.1 (8B)**
   unos 4,9 GB; soporta tools, clave para el paso 5
4. **Claude Code**
   el motor de todas las prácticas del máster

TERMINAL · UNA SOLA VEZ

```
winget install Ollama.Ollama  # o instalador de ollama.com
ollama pull llama3.1  # ~4,9 GB, lanzadlo antes de clase
ollama list  # comprobar que está
```

POR PROYECTO

```
python -m venv .venv
.venv\Scripts\activate  # Windows
pip install -r requirements.txt
```

AL INSTALAR OLLAMA

Os ofrecerá crear una cuenta: no hace falta. Pulsad **'No thanks, I'll use Ollama locally'** y cerrad la ventana: queda funcionando en segundo plano.

thePower Tech School

Paso 1 · Setup del proyecto

QUÉ HACEMOS

- Proyecto + entorno virtual
- requirements.txt: langchain, langchain-community, langchain-ollama
- Comprobar que Ollama tiene llama3.1
- Script mínimo que dice hola al modelo

CONCEPTO CLAVE

Un ChatModel local responde con un mensaje con rol (AIMessage), no con texto suelto. Todo lo demás se construye encima de este motor.



FÍJATE EN

el modelo vive en tu disco (ollama list) y devuelve un AIMessage, no un texto: el parser llega en el paso 2.

thePower Tech School

Paso 1 · Pregunta a la clase

PREGUNTA A LA CLASE

¿Por qué el modelo devuelve un mensaje (AIMessage) y no un simple texto?

Pensad en ChatGPT: ¿Quién dice cada cosa?

thePower Tech School

Paso 2 · Primera cadena LCEL

QUÉ HACEMOS

- PromptTemplate que traduce al inglés
- ChatOllama(llama3.1) como modelo
- StrOutputParser para quedarnos con el texto
- Unir las tres piezas con |

CONCEPTO CLAVE

Tres piezas independientes, prompt, modelo y parser, unidas con |. Cada una se cambia sin tocar las otras dos.



FÍJATE EN

ES TEXTO: True. En el paso 1 salía un AIMessage; la diferencia es el parser trabajando.

thePower Tech School

Paso 2 · Anatomía de la cadena

01_chain.py

```python
"""Paso 2 · Primera cadena LCEL: prompt | modelo | parser."""
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate
from langchain_ollama import ChatOllama

# 1) PROMPT: la instrucción se escribe una vez; {texto} es el hueco que rellenamos al invocar
prompt = ChatPromptTemplate.from_messages([
    ("system", "Eres un traductor profesional. Devuelve SOLO la traducción al inglés, sin comentarios."),
    ("human", "{texto}"),
])

# 2) MODELO: el ChatModel local. Cambiar de proveedor = cambiar solo esta línea
modelo = ChatOllama(model="llama3.1", temperature=0)

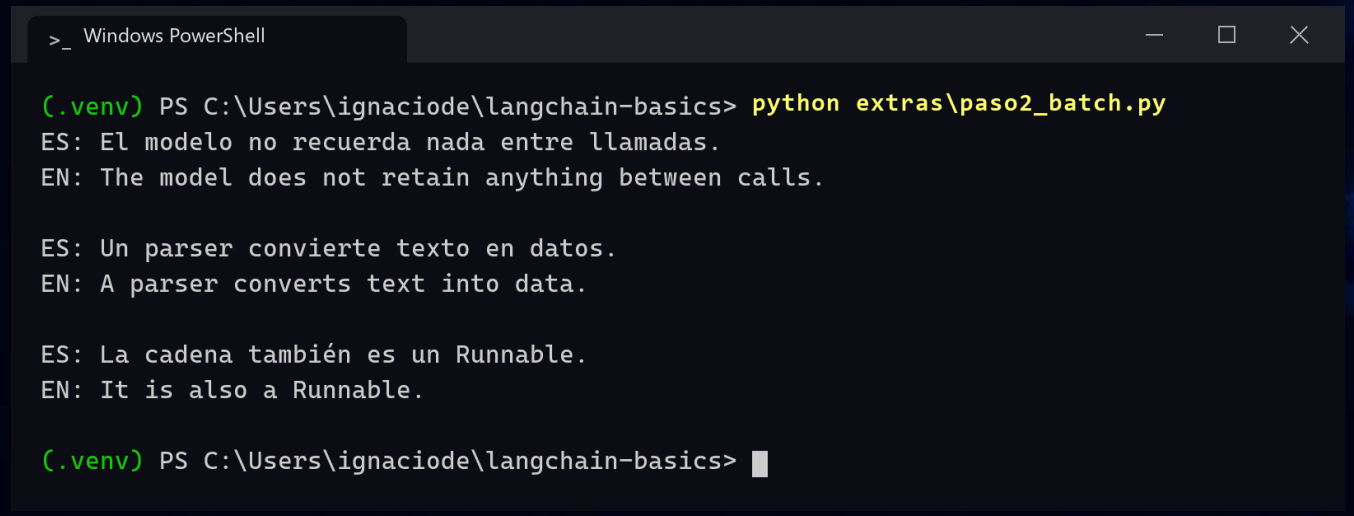
# 3) PARSER: el modelo devuelve un AIMessage; el parser se queda con el texto limpio
parser = StrOutputParser()

# LCEL: el | une las tres piezas en una tubería. La cadena también es un Runnable
cadena = prompt | modelo | parser
```

1. **Prompt**
   la instrucción se escribe una vez; {texto} es el hueco que se rellena al invocar
2. **Modelo**
   cambiar de proveedor es cambiar solo esta línea
3. **Parser**
   convierte el AIMessage en texto limpio
4. **LCEL**
   el | une las piezas; la cadena entera también es un Runnable

thePower Tech School

Paso 2 · Variación: batch



VARIACIÓN EN VIVO

QUÉ CAMBIAMOS

La misma cadena con cadena.batch sobre una lista de 3 frases: tres traducciones sin escribir ningún bucle.

QUÉ DEMUESTRA

La misma cadena procesa una lista entera sin escribir un bucle. Y un aviso real: la tercera traducción se ha comido 'La cadena'. Revisad siempre la salida.

thePower Tech School

Paso 2 · Pregunta a la clase

PREGUNTA A LA CLASE

Si mañana quiero traducir al francés, o usar GPT en vez de llama, ¿qué líneas toco?

Mirad las tres piezas de la cadena.

thePower Tech School

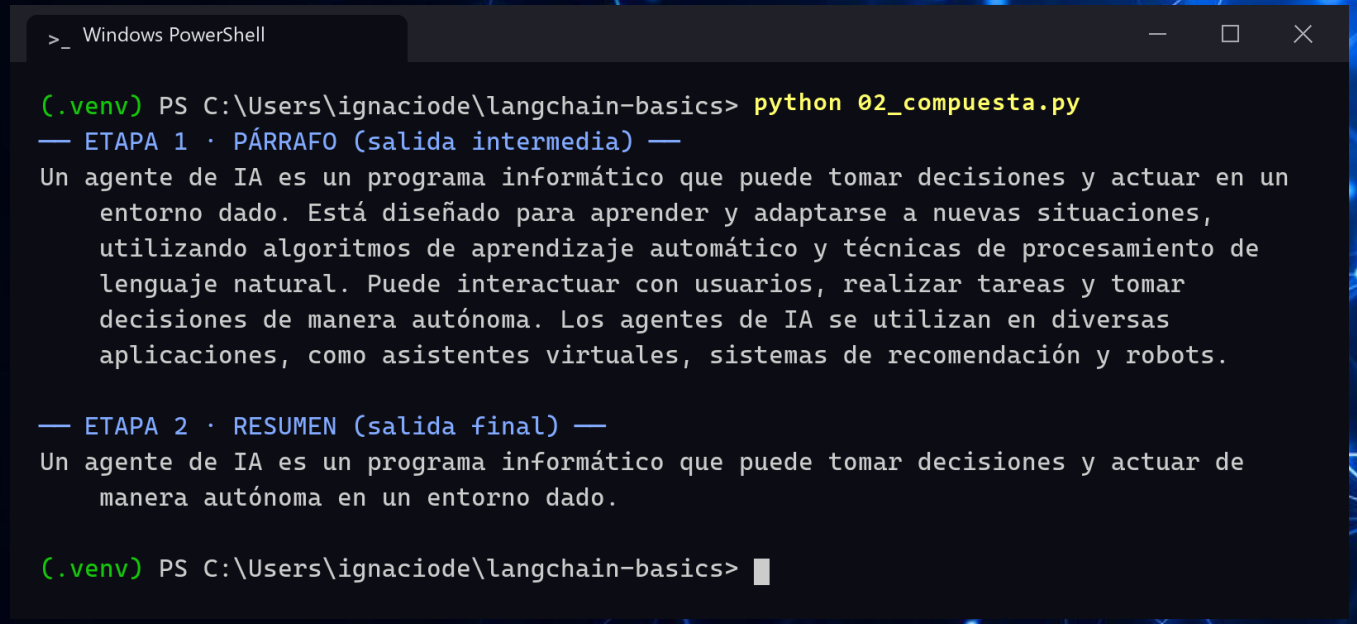
Paso 3 · Cadena compuesta

QUÉ HACEMOS

- Etapa 1: generar un párrafo sobre un tema
- Etapa 2: resumirlo en una frase
- RunnablePassthrough para no perder nada
- Ver salida intermedia y final

CONCEPTO CLAVE

Una etapa por tarea: la salida de una alimenta la siguiente y la intermedia queda a la vista para depurar.



FÍJATE EN

la salida intermedia: si el resumen fuera malo, aquí sabríamos qué etapa falla.

thePower Tech School

Paso 3 · Cómo viaja el diccionario

02_compuesta.py

```python
modelo = ChatOllama(model="llama3.1", temperature=0)
parser = StrOutputParser()

# Etapa 1: genera un párrafo sobre el tema
generar = (
    ChatPromptTemplate.from_template("Escribe un párrafo divulgativo de unas 80 palabras sobre: {tema}")
    | modelo
    | parser
)

# Etapa 2: resume ese párrafo en una frase
resumir = (
    ChatPromptTemplate.from_template("Resume en UNA sola frase el siguiente texto:\n\n{parrafo}")
    | modelo
    | parser
)

# RunnablePassthrough.assign deja pasar la entrada y le AÑADE claves nuevas.
# Así el diccionario va creciendo: {tema} -> {tema, parrafo} -> {tema, parrafo, resumen}
pipeline = (
    RunnablePassthrough.assign(parrafo=generar)
    | RunnablePassthrough.assign(resumen=resumir)
)
```

ENTRADA

`{ tema }`

TRAS 1 · GENERAR

`{ tema, parrafo }`

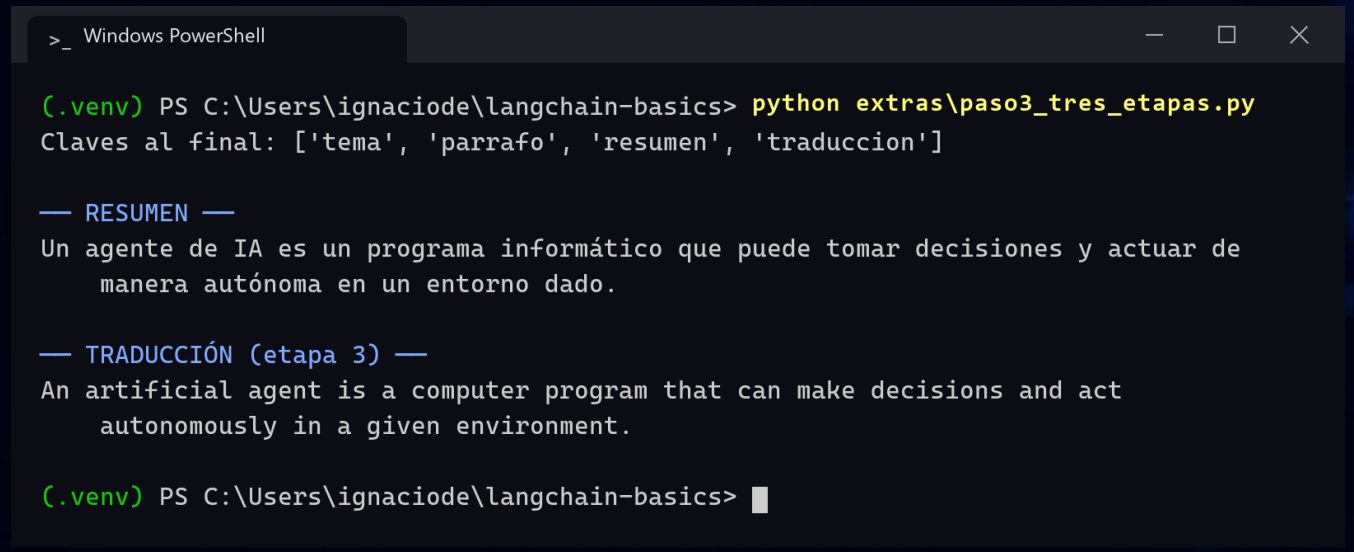
TRAS 2 · RESUMIR

`{ tema, parrafo, resumen }`

3 · *assign deja pasar lo que llega y añade una clave: nadie tira nada.*

thePower Tech School

Paso 3 · Variación: una tercera etapa



VARIACIÓN EN VIVO

QUÉ CAMBIAMOS

Un RunnablePassthrough.assign más, que traduce el resumen al inglés: generar, resumir y traducir.

QUÉ DEMUESTRA

Añadir una etapa es añadir una línea. El diccionario acaba con cuatro claves: generar, resumir y traducir, como en la slide de cadenas compuestas.

thePower Tech School

Paso 3 · Pregunta a la clase

PREGUNTA A LA CLASE

¿Por qué no pedirlo todo en un solo prompt: 'escribe un párrafo y resúmelo'?

¿Qué perderíais cuando el resultado salga mal?

thePower Tech School

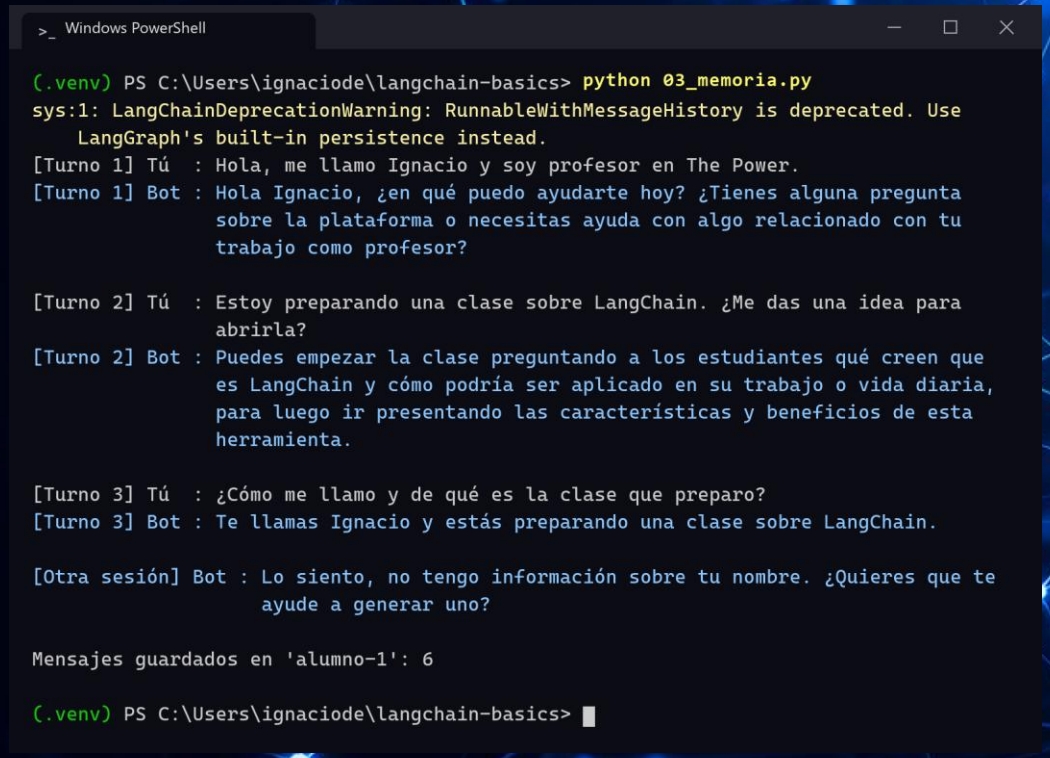
Paso 4 · Añadir memoria

QUÉ HACEMOS

- Envolver la cadena con RunnableWithMessageHistory
- Un historial por session_id
- 3 turnos: recuerda nombre y contexto
- Control: otra sesión no recuerda nada

CONCEPTO CLAVE

El modelo no recuerda: la cadena reenvía la conversación en cada llamada, guardada por session_id.



FÍJATE EN

el turno 3 sabe tu nombre, la otra sesión no. El aviso amarillo no es un error: es el puente a LangGraph.

thePower Tech School

Paso 4 · Dónde vive la memoria

03_memoria.py

```python
prompt = ChatPromptTemplate.from_messages([
    ("system", "Eres un asistente amable. Responde en español y en una o dos frases."),
    MessagesPlaceholder(variable_name="historial"),   # aquí se inyecta la conversación previa  (1)
    ("human", "{pregunta}"),
])
cadena = prompt | ChatOllama(model="llama3.1", temperature=0) | StrOutputParser()

# El "almacén" de memoria: un historial por session_id (en producción: Redis o Postgres)  (2)
historiales: dict[str, InMemoryChatMessageHistory] = {}

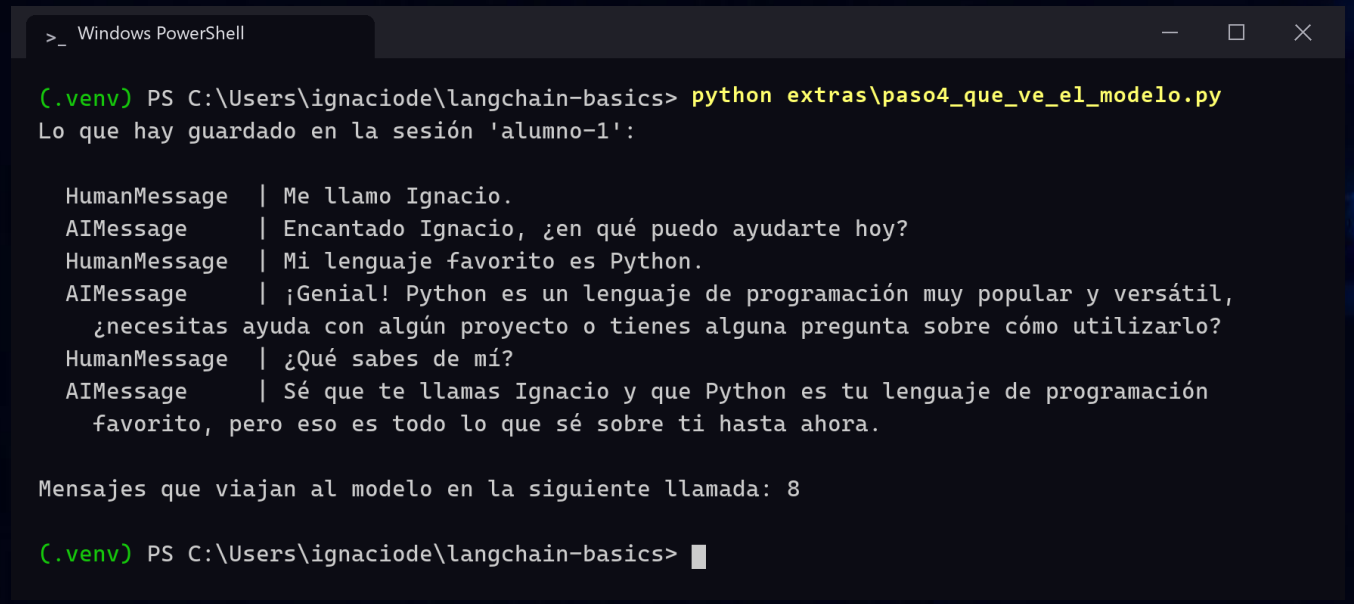
def obtener_historial(session_id: str) -> InMemoryChatMessageHistory:
    if session_id not in historiales:
        historiales[session_id] = InMemoryChatMessageHistory()
    return historiales[session_id]

chat = RunnableWithMessageHistory(  (3)
    cadena,
    obtener_historial,
    input_messages_key="pregunta",
    history_messages_key="historial",
)
```

1. **El hueco**
   MessagesPlaceholder: en el prompt se reserva sitio para la conversación
2. **El almacén**
   un historial por session_id; en producción, Redis o Postgres
3. **El envoltorio**
   antes de cada llamada mete el historial; después guarda pregunta y respuesta

thePower Tech School

Paso 4 · Variación: qué ve realmente el modelo



VARIACIÓN EN VIVO

QUÉ CAMBIAMOS

Tras 3 turnos imprimimos el historial guardado y contamos los mensajes que viajarían en la siguiente llamada.

QUÉ DEMUESTRA

La memoria es una lista que se reenvía entera: 6 mensajes guardados + system + pregunta nueva = 8. Por eso recordar cuesta tokens.

thePower Tech School

Paso 4 · Pregunta a la clase

PREGUNTA A LA CLASE

En el turno 50 de una conversación, ¿cuántos mensajes viajan en cada llamada? ¿Qué problema da?

Recordad la variación: 6 mensajes guardados, 8 enviados.

thePower Tech School

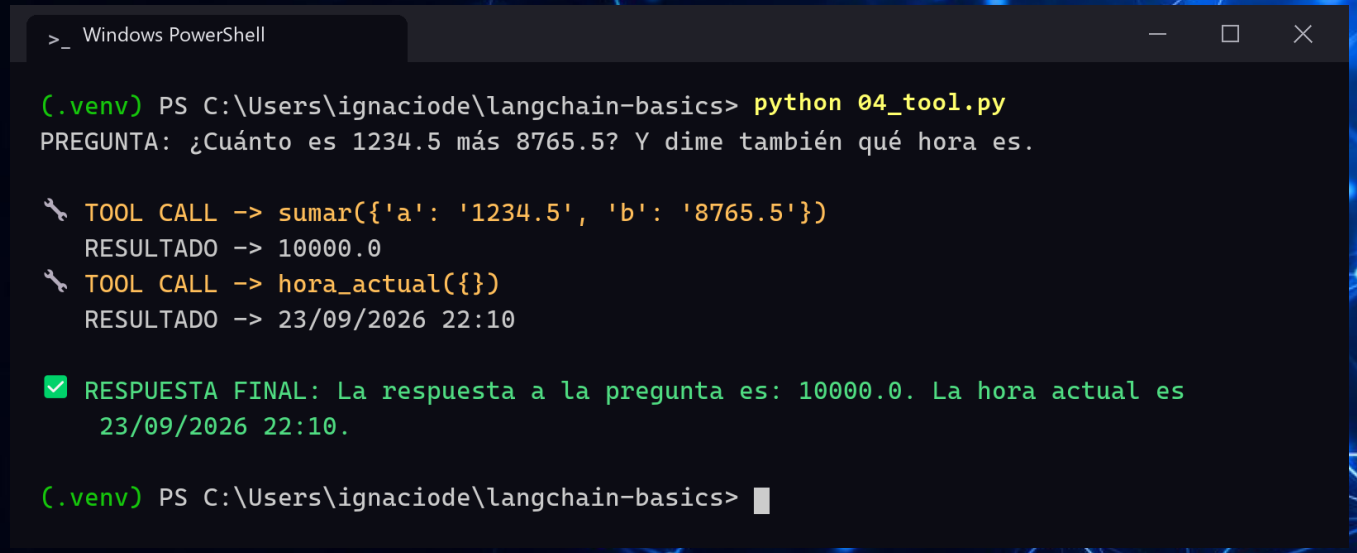
Paso 5 · Añadir una herramienta

QUÉ HACEMOS

- Dos tools con @tool: sumar y hora_actual
- bind_tools: enseñar el menú al modelo
- Pregunta que obliga a usarlas
- Ver la tool call y la respuesta final

CONCEPTO CLAVE

El modelo no ejecuta nada: pide la llamada, nuestro código la ejecuta y le devuelve el resultado.



FÍJATE EN

el modelo no responde: pide. Nuestro código ejecuta, le devuelve el resultado y solo entonces contesta.

thePower Tech School

Paso 5 · El ciclo de una tool call

04_tool.py

```python
@tool
def sumar(a: float, b: float) -> float:
    """Suma dos números y devuelve el resultado."""
    return a + b


@tool
def hora_actual() -> str:
    """Devuelve la fecha y hora actuales del sistema."""
    return datetime.now().strftime("%d/%m/%Y %H:%M")


tools = [sumar, hora_actual]
tools_por_nombre = {t.name: t for t in tools}

# bind_tools: le enseñamos al modelo el "menú" de herramientas (nombre + docstring + argumentos)
modelo = ChatOllama(model="llama3.1", temperature=0).bind_tools(tools)

if __name__ == "__main__":
    pregunta = "¿Cuánto es 1234.5 más 8765.5? Y dime también qué hora es."
    mensajes = [HumanMessage(pregunta)]
    print("PREGUNTA:", pregunta, "\n")

    # 1) El modelo NO responde: pide llamar a las tools
    ai = modelo.invoke(mensajes)
    mensajes.append(ai)
    for llamada in ai.tool_calls:
        print(f"🔧 TOOL CALL -> {llamada['name']}({llamada['args']})")
        # 2) Nuestro código ejecuta la tool y devuelve el resultado como ToolMessage
        resultado = tools_por_nombre[llamada["name"]].invoke(llamada["args"])
        print(f"   RESULTADO -> {resultado}")
        mensajes.append(ToolMessage(content=str(resultado), tool_call_id=llamada["id"]))

    # 3) Con los resultados en el contexto, el modelo redacta la respuesta final
    final = modelo.invoke(mensajes)
    print("\n✅ RESPUESTA FINAL:", final.content)
```

1. **La descripción**
   @tool + docstring + type hints = lo que lee el modelo para decidir

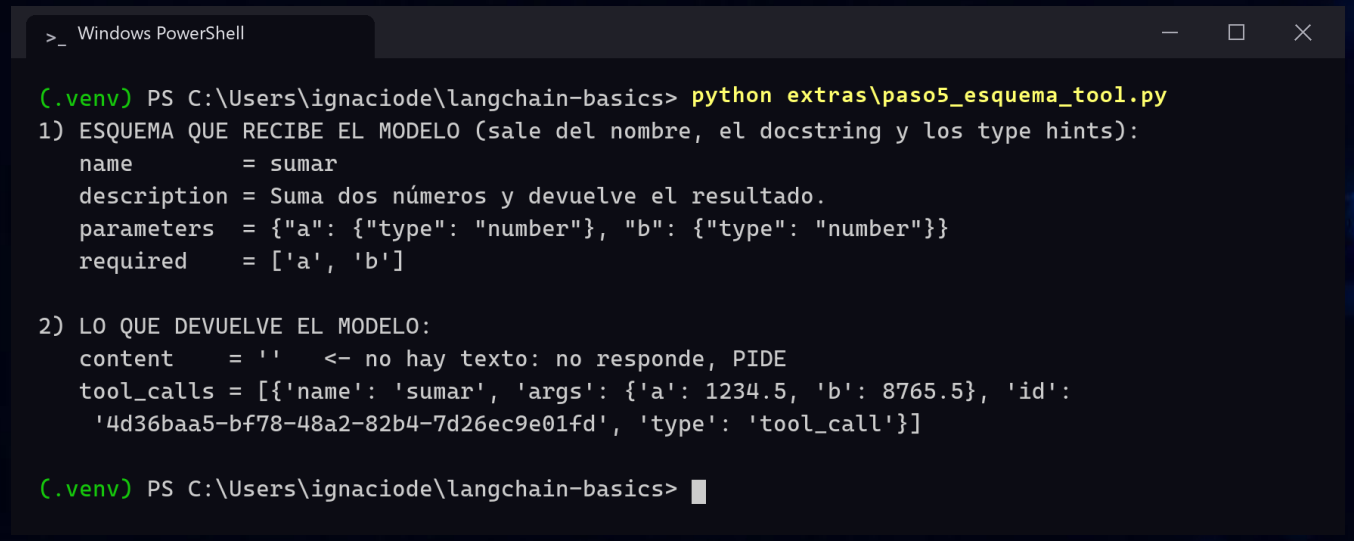
2. **El menú**
   bind_tools pega la lista de tools a cada llamada

3. **La petición**
   ai.tool_calls: el modelo pide, nuestro código ejecuta

4. **La vuelta**
   ToolMessage con el resultado y segundo invoke para la respuesta

thePower Tech School

Paso 5 · Variación: la tool por dentro



VARIACIÓN EN VIVO

QUÉ CAMBIAMOS

Imprimimos el esquema que recibe el modelo de la tool sumar y su respuesta en crudo: content y tool_calls.

QUÉ DEMUESTRA

El docstring se convierte en la descripción y los type hints en el esquema. Y la respuesta llega con content vacío: solo una petición con argumentos.

thePower Tech School

Paso 5 · Pregunta a la clase

PREGUNTA A LA CLASE

¿Cómo supo el modelo que existía una tool llamada sumar y para qué servía?

El modelo no ha visto el código de la función.

thePower Tech School

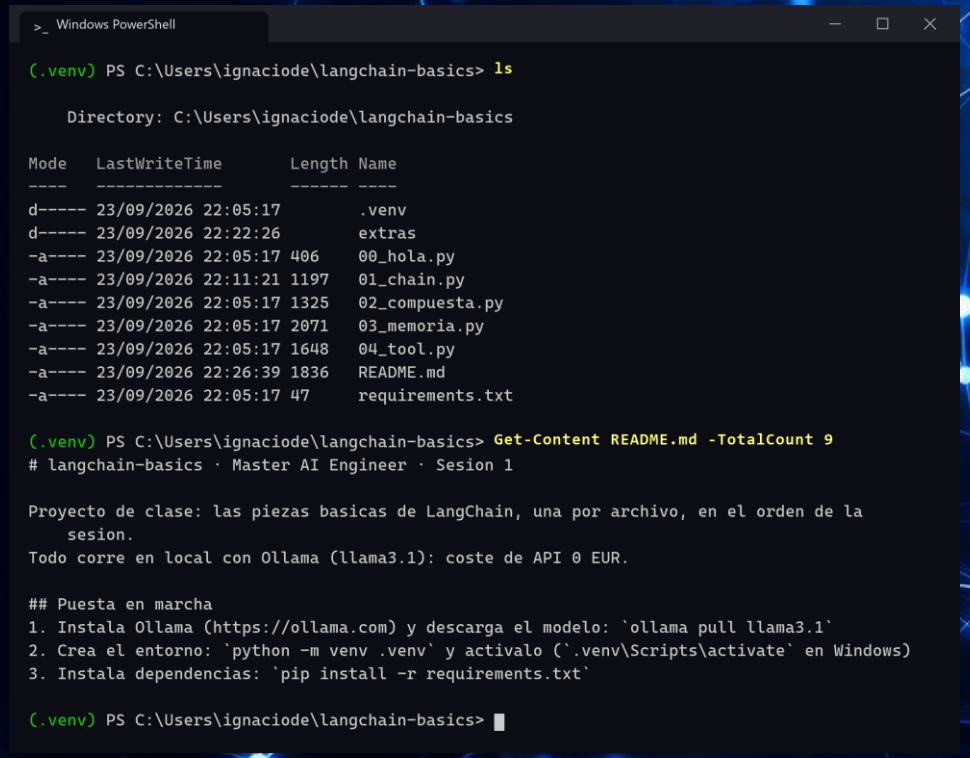
Paso 6 · Cierre y material

QUÉ HACEMOS

- Repasar el código con comentarios
- README que explica cada paso
- Extras con las variaciones de clase
- Repo listo para compartir

CONCEPTO CLAVE

Un repo que solo entiende quien lo escribió no es un entregable: los comentarios y el README son parte del trabajo.



FÍJATE EN

un archivo por paso, numerado en el orden de la clase; es el punto de partida del ejercicio.

thePower Tech School

Recap: de la teoría al código

Caja del mapa	Dónde la hemos usado	La línea clave
Models	00_hola.py	`ChatOllama(model="llama3.1")`
Prompts	01_chain.py	`ChatPromptTemplate.from_messages(...)`
Parsers	01_chain.py	`StrOutputParser()`
Chains · LCEL	01_chain.py · 02_compuesta.py	`prompt | modelo | parser · assign(...)`
Memory	03_memoria.py	`RunnableWithMessageHistory(cadena, ...)`
Tools	04_tool.py	`@tool + modelo.bind_tools(tools)`

La única caja que no hemos tocado: **Retrieval (RAG)**. Tiene sesiones propias: bases de datos, RAG y RAG avanzado.

thePower Tech School

Ejercicio 1 · Tu primer resumidor con LCEL

QUÉ CONSTRUIR

BÁSICO · 1 h

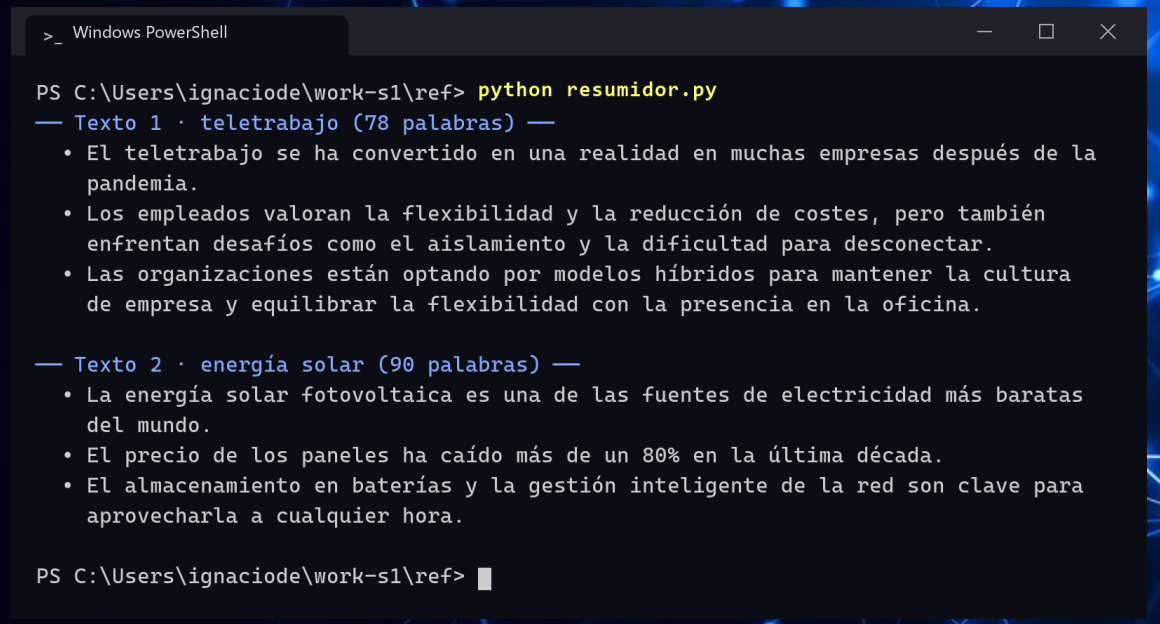
Una cadena LCEL que reciba un texto largo y devuelva un resumen en 3 viñetas, con el patrón **prompt | modelo | parser**.

PASOS SUGERIDOS

1. Prompt con una instrucción clara
2. Conectar un modelo gratis (Ollama o Gemini free tier)
3. Parser que fuerce 3 viñetas
4. Probarlo con 2 textos distintos

ENTREGABLE

Repositorio con el script + captura de las 2 ejecuciones de ejemplo.



ASÍ DEBE VERSE

Ejecución real de una solución de referencia: 2 textos, 3 viñetas cada uno. El código no se enseña: es lo que tienen que construir.

thePower Tech School

Ejercicio 2 · Reto: memoria + herramientas

QUÉ ENTREGAR

RETO

- Partir de 03_memoria.py y 04_tool.py
- Un chat con memoria (session_id) que use al menos una tool
- Conversación de varios turnos que dependa de las dos cosas
- Repo + captura de la conversación con la tool call visible

ASÍ DEBE VERSE (EJEMPLO ORIENTATIVO)

```
Tú  : suma 20 y 22
      TOOL CALL -> sumar(a=20, b=22) -> 42
Bot : 20 + 22 = 42
Tú  : multiplica el resultado anterior por 2
      TOOL CALL -> multiplicar(a=42, b=2) -> 84
Bot : 42 x 2 = 84
```

El segundo turno solo sale bien si recuerda el 42 (memoria) y vuelve a usar una tool.

PISTA

La tool call y el ToolMessage también tienen que quedar en el historial; si no, no recordará el resultado.

PARA NOTA

Una tool que consulte algo real (el tiempo, un CSV propio) o memoria con resumen.

S5

PRÓXIMA SESIÓN · LangGraph

El bucle de tools que hoy escribimos a mano, convertido en un grafo con estado y memoria persistente.

thePower Tech School